Final Project Report

Henry Paproski Johnny Zhang
Faculty of Science, University of Alberta
{hpaprosk, ziqi24}@ualberta.ca

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

1. Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

2. Related Work

The inspiration for this project is the work presented in [1], which introduces a method for generating dense 3D reconstructions by leveraging a sparse base produced using structure-from-motion (SfM). The appeal of this approach is the ability to take a coarse initial model and iteratively refine it into a high accuracy final model. Our implementation focuses on the core pipeline described in this system, specifically the transition from sparse mapping to dense surface estimation through scene-flow updates computed using view-predictive optical flow.

A critical component of monocular reconstruction is the underlying tracking and mapping system. The original system in [1] utilizes Parallel Tracking and Mapping (PTAM) [2] to provide real-time camera poses and a sparse point cloud. However, due to the limited availability of PTAM implementations compatible with Python, we substituted this component with ColMap [3], another SfM library which was readily availanle. While ColMap is typically used for offline processing, it provides the sparse foundation necessary to test the subsequent dense reconstruction stages.

The methods for dense reconstruction from a single camera involved many complex optimization problems. The methods described in [1] frequently utilize Variational methods for global energy minimization, such as the TV-L1 optical flow algorithm discussed in [4] and the denoising technique mentioned in [1]. These methods ensure that the final product both aligns with what's seen in the video while still retaining smoothness. In our implementation, we utilize Poisson reconstruction [6] to generate a base mesh from the initial sparse data, which is then used for further refinement. Although the reference paper [1] emphasizes live performance, our project prioritizes the algorithmic pipeline's ability to enhance model fidelity and photo-consistency by building upon existing SfM data.

3. Methods

Our project implementation is split into seven pipeline stages. First, we use SfM to obtain a sparse point cloud and camera matrices. Second, we generate a base mesh via Poisson reconstruction. Third, we perform keyframe selection to establish bundles for refinement. Fourth, we predict views from the current mesh to analyze discrepancies using optical flow. Fifth, we update mesh vertices based on these flow results to refine the reconstruction. Sixth, we apply a denoising algorithm to ensure spatial smoothness. Finally, we integrate the refined output into a global representation of the entire scene.

3.1. Overview

In this section we walk through each of the stages listed above in the order that the pipeline executes them, detailing both our implementation choices and the places where we deviate from [1]. We illustrate each stage with intermediate outputs from a single running example throughout: a handheld video of a cluttered desk scene of *Henry's desk*.

3.2. Structure from Motion

The goal of the first stage of the pipeline is to recover camera intrinsics, per-frame extrinsics, and ultimately sparse 3D point cloud from the input video. The original system in [1] uses PTAM [2] to obtain these quantities in real-time. Since we had trouble finding a existing workable implimentaiton of PTAM, our implementation uses COLMAP [3] via the `pycolmap` bindings. This provides a well-tested incremental SfM backend. Although COLMAP is a batch method and therefore we have to sacrifice the live aspect of [1], its outputs (calibration matrix, world-to-camera poses, and a

reprojection-filtered sparse point cloud) are exactly what we need for the remainder of the pipeline and so was selected.

In COLMAP's pinhole model, a 3D world point $\mathbf{X}_j \in \mathbb{R}^3$ projects into an image i via

$$\tilde{\mathbf{u}}^i_j = \mathbf{K}[\mathbf{R}_i|\mathbf{t}_i]\tilde{\mathbf{X}}_j, \qquad \mathbf{u}^i_j = \tilde{\mathbf{u}}^i_j / \tilde{u}^i_{j,3}, \tag{1}$$

where $\tilde{\cdot}$ denotes homogeneous coordinates. The incremental mapper [3] refines the intrinsics, per-frame extrinsics, and 3D structure by minimizing the reprojection error over all observations $(i,j) \in \mathcal{O}$,

$$\min_{\mathbf{K},\{\mathbf{R}_i,\mathbf{t}_i\},\{\mathbf{X}_j\}} \sum_{(i,j)\in\mathcal{O}} \rho(\|\mathbf{u}^i_j - \hat{\mathbf{u}}^i_j\|^2), \tag{2}$$

where $\hat{\mathbf{u}}^i_j$ is the observed feature location and $\rho(\cdot)$ is a loss that down-weights outliers from incorrect matches [3]. The output of this is the $(\mathbf{K}, \mathbf{R}_i, \mathbf{t}_i, \mathbf{X}_j)$ that the remainder of our pipeline needs.

Given a directory of frames, we run feature extraction under this pinhole camera model and perform sequential matching. When multiple reconstructions are produced we retain the one containing the largest number of registered images. Following this we call COLMAP's image undistortion routine so that all subsequent stages may assume a pure pinhole projection, which is an assumption implicit in both the view-prediction of Section 3.3 and the projection Jacobian of Section 3.4 of [1]. The undistorted reconstruction is then parsed into a `CameraModel` holding K and image dimensions, a list of `Keyframes` storing world-to-camera pose (R, t) and the 3×4 projection matrix $P = K[R|t]$, and a list of `SparsePoints`.

To remove invalid triangulations that would otherwise corrupt the base mesh, we filter the COLMAP point cloud by discarding all points whose track length is too low or whose mean reprojection error is too high. Note that the point cloud produced by COLMAP is unoriented but the Poisson reconstruction used in the next stage requires consistently oriented surface normales. This means that we have to estimate a normal at each surviving point by averaging the (negated) viewing directions of every camera that observes it. This should guarantee that the resulting normal points back toward the cameras that saw the surface and thus faces outward from the object. An example of the resulting sparse point cloud is shown below (Figure 1).

3.3. Base Mesh Construction

With a sparse oriented point cloud in hand, we next fit a smooth initial surface that will be used in the scene flow refinement. Newcombe and Davison [1] construct

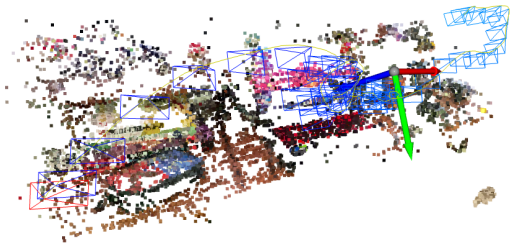


Figure 1: Sparse point cloud produced by COLMAP on *Henry's desk* after filtering.

their base surface by fitting a multi-scale compactly-supported radial basis function (MSCSRBF) implicit [5] to the oriented SfM points and polygonising its zero level set. MSCSRBF is effective because it combines the global consistency of a single implicit function with the locality of compactly-supported basis functions, which makes it well-suited to the sparse, non-uniformly distributed point clouds produced by PTAM. It is also cheap enough to re-fit each time a new keyframe is added which support their live situation. In our implementation we instead use Poisson surface reconstruction [6], which similarly solves globally for an implicit indicator function $\chi : \mathbb{R}^3 \to \mathbb{R}$ whose gradient matches the oriented point normals. Treating the input samples $\{(\mathbf{X}_j, \mathbf{n}_j)\}$ as discrete samples of a vector field $\vec{\mathbf{V}}$ whose value at each point is the surface normal, Kazhdan et al. observe that the indicator gradient should satisfy $\nabla\chi \approx \vec{\mathbf{V}}$. Taking the divergence of both sides and solving in the least-squares sense yields the Poisson equation

$$\Delta\chi = \nabla \cdot \vec{\mathbf{V}}, \tag{3}$$

which is discretized over an adaptive octree and solved with a multigrid relaxation. The reconstructed surface is then extracted as the iso-level set $\{\mathbf{x} : \chi(\mathbf{x}) = c\}$. The iso-value, c, is chosen so that the level set passes through exactly the average of the input samples. This whole process yields a triangle mesh.

The two methods occupy the same role in the pipeline but Poisson was the more practical choice as it is available as a one line function call in Open3D, whereas we were unable to find a easily workable MSCSRBF implementation compatible with our Python pipeline. Since we do not target live operation, the per-keyframe refit efficiency that motivated MSCSRBF in [1] is not a major concern for us.

The raw Poisson output inevitably contains hallucinated geometry in regions far from the input samples, since the method extrapolates the implicit function over its entire support. We therefore apply a sequence of cleanup steps before handing the mesh off to the view-

prediction stage. First, Poisson's per-vertex density estimate is used to trim low-confidence vertices whose density falls below a quantile threshold, which removes thin sheets in unsampled regions. Second, the mesh is cropped to a padded bounding box of the point cloud. This eliminates any remaining extrapolated geometry outside the observed scene. Next, we cluster connected components and only keep the largest one, discarding any small islands that were produced. Finally, we apply some small number of Taubin smoothing iterations to smooth out the jagged "octree" look of the Poisson output without shrinking the mesh, and fix the triangle winding so that all faces are oriented consistently.

In order to interface with the next (view-prediction) stage, the cleaned mesh is loaded into an Open3D raycasting scene once and reused across all keyframes. So given a camera pose and intrinsics, we generate a ray bundle and cast it against the mesh, recording each pixel the Euclidean distance from the camera center to the intersection. Pixels whose rays miss the surface (for example, background regions or holes left by the cleanup above) are marked with zero depth and propagate to the view prediction as invalid so they are later excluded from the scene flow update. The resulting depth maps form the direct input to the view-predictive optical flow stage described next. An example of the base mesh produced by this pipeline is shown in Figure 2.

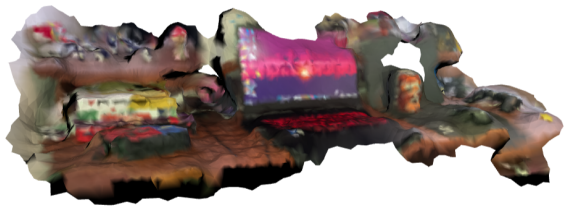


Figure 2: Base mesh produced by Poisson surface reconstruction on *Henry's desk* after cleanup.

3.4. Bundle Selection

The subsequent refinement stages operates on a *bundle* each consisting of one reference frame and n comparison frames. The reference supplies the depth map that is to be refined, while the comparisons provide the additional views whose optical flow drives the scene flow update. The choice of these frames is important: too few comparisons or a poor distribution of baselines leads directly to the aperture problem in the flow computation, while redundant comparisons waste computation without improving the conditioning of the linear system.

Newcombe and Davison [1] solve this problem online using a rolling buffer of the most recent 60 frames. A new reference frame is initialised whenever the co-visibility V_r of the previous reference with the current camera drops below a threshold $\epsilon_r = 0.3$, where V_r is computed by rendering the base surface from both poses and measuring the fraction of reference pixels that remain visible. For each new reference, they then wait until the co-visibility V_c of a subsequent frame with the reference drops below $\epsilon_c = 0.7$, at which point the last 60 frames are searched for the n comparisons whose translations are most evenly distributed around the reference view direction, which maximises the baseline of the bundle and helps mitigate the aperture problem in the subsequent dense flow computation.

We follows the same structure in our implimentation: begin by walking through the keyframes in order and running the same co-visibility test as [1]. For each candidate we render the base mesh from both its pose and the pose of the current reference, and measure the fraction of reference-frame pixels whose rays hit the surface and are also hit from the candidate view. A candidate becomes a new reference whenever this overlap falls below $\epsilon_r = 0.3$, matching the paper's threshold. Candidates whose coverage of the base mesh is itself below 10% are skipped outright, as they cannot meaningfully contribute to any refinement.

For each selected reference, we then form a buffer of co-visible neighbours and choose n comparisons from it. Rather than maximising raw translational disparity, we make the angular criterion explicit: given the reference viewing direction $\mathbf{d}$, we project each candidate's translation vector $\mathbf{t}_c - \mathbf{t}_{ref}$ onto the plane perpendicular to $\mathbf{d}$ and record its azimuth $\varphi \in [0, 2\pi)$. Candidates whose baseline lies outside a minimum and maximum range, or whose own view direction deviates from $\mathbf{d}$ by more than 20°, are discarded so that the comparison cameras look at roughly the same portion of the scene as the reference. We then select the n comparisons greedily: the first is the one with the largest baseline, and each subsequent comparison is chosen to maximise the smallest circular distance in φ to all already-selected comparisons. This recovers the "distributed around the reference view" property from [1] in a form that is cheap to evaluate. The resulting bundles are consumed by the view-prediction and scene-flow stages described next.

Figure 3: Approximate reference frames for *Henry's desk* in our pipeline.

3.5. View-Predictive Optical Flow

3.5.1. View-Prediction

The purpose of the view prediction stage is to generate a virtual image that represents how a scene should appear from the perspective of a specific comparison camera, given our current 3D reconstruction. Following the methodology in [1], this "synthetic" view serves as a intermediate step for computing view-predictive optical flow. By comparing the predicted image with the actual captured image from the comparison pose, the system can identify discrepancies caused by inaccuracies in the base mesh.

Unlike the implementation described in [1], wherein the reference frame image is back-projected onto the base model, then said textured model is projected into the reference. We instead use the base mesh to obtain a depth map for the comparison frame and use it to restore the 3D coordinates where each ray in the image intersects the base mesh and project those points into the reference frame. Once we do that we have a map for which pixel in the reference corresponds to a given pixel in the comparison so we perform a sampling to obtain our final image prediction. This process is shown in more detail in algorithm 1.

Algorithm 1 View-Predictive Image Synthesis

$K_{inv} \leftarrow K^{-1}$
for each pixel $\mathbf{u} = (x, y)$ in comparison frame **do**
  $\mathbf{r} \leftarrow K_{inv} \cdot \tilde{\mathbf{u}}$ ▷ Back-project to camera ray
  $\hat{\mathbf{r}} \leftarrow \mathbf{r}/\|\mathbf{r}\|$
  $X_{comp} \leftarrow \hat{\mathbf{r}} \cdot D(\mathbf{u})$ ▷ Scale by Euclidean depth
  $X_{world} \leftarrow R_{comp}^{-1}(X_{comp} - t_{comp})$
  $X_{ref} \leftarrow R_{ref} \cdot X_{world} + t_{ref}$
  $\tilde{\mathbf{u}}_{ref} \leftarrow K \cdot X_{ref}$ ▷ Project into reference
  $\mathbf{u}_{ref} \leftarrow \tilde{\mathbf{u}}_{ref}/X_{ref}[2]$ ▷ Dehomogenise
  if $\mathbf{u}_{ref}$ inside image bounds **and** $X_{ref}[2] > 0$ **then**
    $I_{pred}(\mathbf{u}) \leftarrow I_{ref}(\mathbf{u}_{ref})$
  else
    $I_{pred}(\mathbf{u}) \leftarrow 0$ ▷ Mark invalid
  end if
end for
return I_{pred}

Unfortunately, in cases where the pixels land outside of the image boundary or there is no base mesh we are unable to extract any information about said pixel and simply set it to zero as seen in figure 1. To keep track of these pixels we actually construct a mask of valid pixels, which simply tells us which pixels didn't intersect the base mesh and which pixels mapped to someplace that was outside of the reference image boundaries.

Figure 4: An Image prediction generated using algorithm 1

3.5.2. Optical Flow

Dense correspondence between the reference and comparison frames is the core mechanism by which the base mesh is refined. Following Newcombe and Davison [1], we compute optical flow not between raw image pairs, but between a synthesized view prediction and the true comparison image. Because the view prediction already accounts for the rotational component of camera motion, the residual flow field measures only the geometric error in the current mesh estimate, dramatically reducing the displacement magnitudes that the flow algorithm must handle.

The flow is computed by minimizing the TV-L^1 energy function [7]:

$$\mathbf{E}_{\mathbf{u}} = \int_{\Omega} \lambda |I_0(\mathbf{x}) - I_1(\mathbf{x}+\mathbf{u}(\mathbf{x}))| \, d\mathbf{x} + \int_{\Omega} |\nabla \mathbf{u}| \, d\mathbf{x} \quad (4)$$

where I_0 and I_1 are the image pair, $\mathbf{u}$ is the displacement field, and λ provides a trade-off between the data fidelity and regularization term. The data term provides robustness to image data errors, and the regularization allows flow discontinuities at occlusion boundaries while producing smooth fields elsewhere [1].

The minimization of this function is done using the method described in [7]. Wherein they define an auxiliary variable $\mathbf{v}$, and use the following convex approximation to solve for d dimensional TV-L^1:

$$\mathbf{E}_{\theta} = \int_{\Omega} \{ \sum_{d} |\nabla u_d| + \sum_{d} \frac{1}{2\theta} (u_d - v_d)^2 + \lambda |\rho(v)| \} \quad (5)$$

Where θ is a small constant such that v is a sufficiently close approximation of u and ρ denotes the image residual. Additionally, in the case of optical flow, $\mathbf{u}$ only has 2 dimensions and thus d in this case is 2. From this convex approximation, a solution can be obtained by performing alternating updates to $\mathbf{u}$ and $\mathbf{v}$.

To update $\mathbf{v}$ let $\rho(\mathbf{u}) = I_1(\mathbf{x} + \mathbf{u}_0) + \nabla I_1 \cdot (\mathbf{u} - \mathbf{u}_0) - I_0(\mathbf{x})$ and define the threshold $\kappa = \lambda\theta|\nabla I_1|^2$. The point-wise update for $\mathbf{v}$ is then:

$$\mathbf{v} = \mathbf{u} + \begin{cases} \lambda\theta\nabla I_1 & \text{if } \rho(\mathbf{u}) < -\kappa \\ -\lambda\theta\nabla I_1 & \text{if } \rho(\mathbf{u}) > \kappa \\ -\rho(\mathbf{u})\frac{\nabla I_1}{|\nabla I_1|^2} & \text{if } |\rho(\mathbf{u})| \leq \kappa \end{cases} \tag{6}$$

Once $\mathbf{v}$ is updated, we then perform an update for $\mathbf{u}$ using a dual formulation, with a dual variable $\mathbf{p}$, as follows:

$$\mathbf{p}^{k+1} = \frac{\mathbf{p}^k + (\tau/\theta)\nabla\mathbf{u}^k}{1 + (\tau/\theta)|\nabla\mathbf{u}^k|} \tag{7}$$

$$\mathbf{u}^{k+1} = \mathbf{v} + \theta\text{div}(\mathbf{p}^{k+1}) \tag{8}$$

where $\tau \leq 1/8$ is the step size. This procedure is only valid for small displacements, and thus is embedded within a coarse-to-fine image pyramid, which will help in avoiding convergence to unfavorable local minima.

3.5.3. Structure-Texture Decomposition

A known limitation of intensity-based optical flow is its sensitivity to illumination artifacts such as shadows and shading reflections, which violate the brightness constancy assumption. To address this, we apply a structure-texture decomposition to both the predicted and comparison images before computing flow. The decomposition separates each image into a structural component I_S, corresponding to the large-scale geometry of the scene, and a texture component $I_T = I - I_S$, containing fine-scale detail from which illumination artifacts have been largely removed as seen in figure 2. As shown in Wedel et al. [7], computing flow on the texture component yields substantially more accurate estimates in the presence of shadows and reflections. In our implementation, this decomposition is computed using a skimage library, and the texture images are passed to the flow computation in place of the originals.

3.5.4. Forward-Backward Consistency

Optical flow estimation is inherently unreliable in occluded regions, where a pixel visible in one frame has no correspondence in the other. To discard such unreliable flow vectors before they influence the scene flow update, we apply a forward-backward consistency check. For each pixel, we compute the forward flow $\mathbf{u}_{fwd}$ from the predicted image to the comparison image, and the backward flow $\mathbf{u}_{bwd}$ in the reverse direction. When there is no occlusion, and the optic flow is accurate the backward flow should point in the opposite direction of the forward flow: $\mathbf{u}_{fwd}(\mathbf{x}) + \mathbf{u}_{bwd}(\mathbf{x} + \mathbf{u}_{fwd}(\mathbf{x})) = 0$.

Figure 5: An Image prediction generated using algorithm 1

Let $\mathbf{w}(\mathbf{x}) = \mathbf{u}_{bwd}(\mathbf{x} + \mathbf{u}_{fwd}(\mathbf{x}))$, then a pixel's flow is marked valid only if this value falls below an adaptive threshold proportional to the local flow magnitude as follows:

$$|\mathbf{u}_{fwd}(\mathbf{x}) + \mathbf{w}(\mathbf{x})|^2 < 0.01(|\mathbf{u}_{fwd}|^2 + |\mathbf{w}(\mathbf{x})|^2) + 0.5 \tag{9}$$

This approach, motivated by the occlusion reasoning in [8], ensures that flow vectors in occluded or poorly constrained regions, such as those which are close to black regions generated by holes in the base mesh, do not contribute to λ_j, which is done by adding these outlier indexes to the mask generated in section 3.1.1.

3.6. Scene Flow

Given the Dense correspondences obtained in the previous section, we now describe how they are used to deform the base mesh into a more accurate depth map. The core idea, illustrated in Figure 3, is that a small 3D displacement $\Delta\mathbf{x}_j$ of a surface point $\mathbf{x}_j$ induces a proportional image displacement $\Delta\mathbf{u}_j^i$ in each camera i. For small displacements, this relationship can be approximated using a first-order Taylor expansion of the projection function $\mathbf{P}^i$ around $\mathbf{x}_j$:

$$\Delta\mathbf{u}_j^i = \mathbf{J}_{\mathbf{x}_j}^i \Delta\mathbf{x}_j \tag{10}$$

where $\mathbf{J}_{\mathbf{x}_j}^i$ is the Jacobian of the projection evaluated at $\mathbf{x}_j$:

$$\mathbf{J}_{\mathbf{x}_j}^i \equiv \left.\frac{\partial\mathbf{P}^i}{\partial\mathbf{x}}\right|_{\mathbf{x}_j} = \begin{bmatrix} \frac{\partial P_u^i}{\partial x} & \frac{\partial P_u^i}{\partial y} & \frac{\partial P_u^i}{\partial z} \\ \frac{\partial P_v^i}{\partial x} & \frac{\partial P_v^i}{\partial y} & \frac{\partial P_v^i}{\partial z} \end{bmatrix}_{\mathbf{x}_j} \tag{11}$$

The optical flow $\Delta\mathbf{u}_j^i$ for each comparison camera i is obtained from the view-predictive flow computed in the previous stage, giving us up to $2n$ constraints per surface point, where n is the number of comparison frames.

Unfortunately, directly solving for the full 3D update $\Delta\mathbf{x}_j \in \mathbb{R}^3$ from these image-space measurements is problematic, as there are infinitely many possible 3D displacements that produce the same projected change.

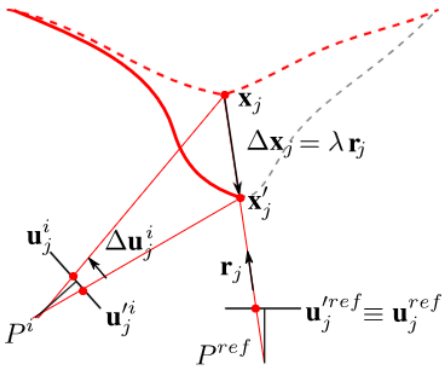


Figure 6: Geometry of our scene flow update. All updates to the base surface vertices are constrained along viewing rays of the reference camera.

3.6.1. Ray Constraint

To rectify this issue, we follow [1] in restricting all surface updates to lie along the viewing ray from the reference camera. Since any deformation of a point $\mathbf{x}_j$ that stays on the same reference ray produces no change in its reference-frame projection, this restriction results in following constraint for the scene flow vector:

$$\Delta\mathbf{x}_j = \mathbf{r}_j\lambda_j \tag{12}$$

where $\lambda_j \in \mathbb{R}$ is a scalar depth update and $\mathbf{r}_j$ is the unit direction of the reference ray:

$$\mathbf{r}_j = \frac{\mathbf{R}^{ref}_{cw}\,[\mathbf{u}^{ref\,T}_j \mid 1]}{\|\mathbf{R}^{ref}_{cw}\,[\mathbf{u}^{ref\,T}_j \mid 1]\|} \tag{13}$$

Substituting this constraint back into the scene flow equation reduces the number of unknowns from three to one per vertex:

$$\Delta\mathbf{u}^i_j = \mathbf{K}^i_j\lambda_j, \qquad \mathbf{K}^i_j \equiv \mathbf{J}^i_{\mathbf{x}_j}\mathbf{r}_j \tag{14}$$

where $\mathbf{K}^i_j \in \mathbb{R}^2$ is the projection of the ray through the Jacobian for camera i.

3.6.2. Solving for the Depth Update

Stacking the two-component equations from all n comparison cameras gives an overdetermined linear system in λ_j:

$$\begin{bmatrix} K^{[u]\,1}_j \\ K^{[v]\,1}_j \\ \vdots \\ K^{[u]\,n}_j \\ K^{[v]\,n}_j \end{bmatrix} \lambda_j = \begin{bmatrix} \Delta u^1_j \\ \Delta v^1_j \\ \vdots \\ \Delta u^n_j \\ \Delta v^n_j \end{bmatrix} \tag{15}$$

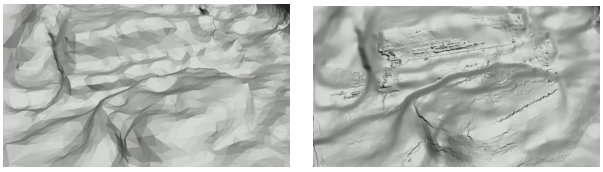
While this could in general be solved by least-squares matrix factorization methods such as QR which would be more numerically stable, we instead exploit the scalar nature of λ_j to obtain the closed-form normal equation solution:

$$\lambda_j = \frac{\sum_{a=1}^{2n} k_a u_a}{\sum_{a=1}^{2n} k_a^2} \tag{16}$$

where k_a and u_a are the scalar elements of $\mathbf{K}_j$ and $\Delta\mathbf{u}_j$ respectively. The primary reason we use this formulation in our implementation because it allows us to apply the valid pixel mask from Section 3.1 and the forward-backward consistency mask from Section 3.1.4 by simply skipping invalid pixels when we perform the accumulation of the numerator and denominator. The updated world point is then:

$$\mathbf{x}'_j = \mathbf{x}_j + \mathbf{r}_j\lambda_j \tag{17}$$

and the new depth map is recovered as the Euclidean distance from the reference camera center to each updated point, which is then used to obtain a new mesh as shown in Figure 4 which illustrates the deformation of the base mesh after one scene flow update.



(a) Base mesh before scene flow

(b) Mesh after scene flow update

Figure 7: Deformation of the base mesh by a single scene flow update.

3.6.3. Iterative Refinement

Because the linearization of the projection function is only valid for small displacements, the update derived above is an approximation that improves as $\mathbf{x}_j$ approaches the true surface. Additionally, the formulation above can also be derived using Gauss-Newton, meaning that this process is a non-linear optimization problem. We therefore iterate the full process: after each

depth update, the new depth map is denoised and retriangulated into a mesh, and optical flow is recomputed between a fresh view prediction from the updated mesh and each comparison image. This produces new, smaller flow residuals that drive a further correction. Following [1], this outer loop runs for up to three iterations, or until the mean reprojection error falls below a certain threshold. In order to ensure that the depth map remains smooth while not falsely inducing smoothness along sharp edges, we apply the TV-L^1 described in the following section to denoise after each scene flow update.

3.7. Global Integration

The scene flow stage produces a refined depth map for a bunch of bundles, but our final goal is a single mesh covering the entire scene. In order to achieve this, we need to convert each depth map into a triangle mesh and then merge them into a running global model. We need to do this without duplicating geometry where (consecutive) bundles overlap.

3.7.1. Per-Bundle Triangulation

We are given a depth map $\mathbf{D}$ from a (reference) pose (R_{ref}, t_{ref}). From this we can recover a 3D vertex for each (valid) pixel by back-projecting along its reference ray (Eq. (9) of [1]):

$$\mathbf{v}_j = \mathbf{D}(\mathbf{u}_j)\mathbf{r}_j + \mathbf{t}_{ref}, \tag{18}$$

here $\mathbf{r}_j$ is the (unit) reference ray (Section 3.5.1 of [1]). Now we can triangulate: each 2×2 block of valid pixels is split into two triangles along its diagonal. This procduces something that is dense and consistent across bundles. Per-vertex normals are then obtained from central finite differences of the back-projected grid: $\mathbf{n}_j = \Delta\mathbf{v}_x \times \Delta\mathbf{v}_y$. This is the same construction as [1].

3.7.2. Surface Error Filtering

The scene flow update additionally produces two error measures at every pixel: 1. value error E^v which is the residual of the linear system the depth update was the solution to, and 2. a surface error E^s that measures reprojection error of the (updated) point in the comparison cameras. We follow [1] and drop vertices whose errors measure above point to the (local) solve as being unreliable. Concretely, we keep vertices using:

$$\neg(E^v_j < \tau_v \wedge E^s_j > \tau_s), \tag{19}$$

which is the inverse of the reject condition implemented in our code. We set our thresholds to $\tau_v = 0.9$ and $\tau_s = 0.001$. Faces whose three vertices do not all survive the filter are removed. Optionally, we also support a variant of the surface threshold in which τ_s is set to the 75th percentile of the valid surface errors observed in the current bundle, which could make the filter more robust.

3.7.3. Fusion

Once a per-bundle mesh has been triangulated and filtered, we fuse it into the running global model in the following way. Not that naively concatenating meshes would leave duplicated and slightly inconsistent geometry wherever multiple bundles overlap. [1] avoid this by removing from each new mesh all vertices whose distance to the integrated surface is below a small threshold ϵ_{dist}. We adopt the same strategy.

Finally, from the bundle's reference pose, we render the (current) global model using Open3D. This produces a z-depth map $z_{glob}(\mathbf{u})$ that is aligned to the reference image grid. Each new vertex $\mathbf{v}_j$ is then projected into the reference view: $\mathbf{u}_j = \pi(K, R_{ref}, t_{ref}, \mathbf{v}_j)$. We drop whenever there is already nearby geometry along the same viewing ray:

$$\begin{aligned} \text{discard } \mathbf{v}_j \quad \text{iff} \quad & z_{glob}(\mathbf{u}_j) > 0 \\ & \wedge\, |z_j - z_{glob}(\mathbf{u}_j)| < \epsilon_{dist}, \end{aligned} \tag{20}$$

where z_j is z-coordinate of $\mathbf{v}_j$ (in camera-frame). We discrard Vertices that either project outside the image bounds or that fall behind the reference camera. The surviving vertices and their faces are appended to the global vertex and face arrays, yielding the next iteration of the global model. We use $\epsilon_{dist} = 0.01$ in scene units, which is comparable to the per-vertex baseline of the bundles in our datasets.

One small deviation from [1] is that we run a few Taubin smoothing iterations on each bundle's mesh before fusing it, which should clean up the high-frequency noise from the triangulation without shrinking the mesh. This is done locally, so it does not propagate across bundles. After every bundle has been integrated, the resulting global model is exported as a PLY file for evaluation.

4. Experiments

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

4.1. Dataset and Setup

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

4.2. Results

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

5. Conclusions

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

References

[1] R. A. Newcombe and A. J. Davison. Live dense reconstruction with a single moving camera. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2010.

[2] G. Klein and D. Murray. Parallel tracking and mapping for small AR workspaces. *Proceedings of the International Symposium on Mixed and Augmented Reality (ISMAR)*, 2007.

[3] J. L. Schönberger and J. M. Frahm. Structure-from-motion revisited. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] C. Zach, T. Pock, and H. Bischof. A duality based approach for realtime TV-L1 optical flow. *Proceedings of the Joint Pattern Recognition Symposium*, 2007.

[5] Y. Ohtake, A. Belyaev, and H.-P. Seidel. A multi-scale approach to 3D scattered data interpolation with compactly supported basis functions. *Proceedings of Shape Modeling International*, 2003.

[6] M. Kazhdan, M. Bolitho, and H. Hoppe. Poisson surface reconstruction. *Proceedings of the Eurographics Symposium on Geometry Processing (SGP)*, 2006.

[7] A. Wedel, T. Pock, C. Zach, H. Bischof, and D. Cremers. An improved algorithm for TV-L1 optical flow. In *Proceedings of the Dagstuhl Seminar on Statistical and Geometrical Approaches to Visual Motion Analysis*, pages 23–45, 2009.

[8] N. Sundaram, T. Brox, and K. Keutzer. Dense point trajectories by GPU-accelerated large displacement optical flow. In *Proceedings of ECCV*, pages 438–451, 2010.